

Tut 05

Anne

Junit good to know

- **@DisplayName:** ändert den angezeigten Namen eines Tests (Standard ist der Methodename)
- **@BeforeAll:** führt eine Methode **einmal vor allen Tests** in dieser Klasse aus (muss zur Klasse gehören, also static sein)
- **@BeforeEach:** führt eine Methode **vor jedem einzelnen Test** aus
- **@Order(int num):** legt fest, in welcher Reihenfolge die Tests ausgeführt
@TestMethodOrder(MethodOrderer.OrderAnnotation.class) das muss dann über der Klasse stehen
- `assertEquals(expected, Methode)`
- `assertTrue(value)`
- `assertFalse(value)`
- `assertNull(value)`

Example: JUnit

```
public class CalculatorTest {
```

```
    private Calculator calc;
```

```
    @BeforeEach
```

```
    void init() {
```

```
        calc = new CalculatorImpl();
```

```
    }
```

```
        @Test
```

```
        @DisplayName("1+2=3")
```

```
        void testAdd() {
```

```
            assertEquals(3, calc.add(1, 2));
```

```
        }
```

Aufgabe

- **@DisplayName:** ändert den angezeigten Namen eines Tests (Standard ist der Methodennamen)
- **@BeforeAll:** führt eine Methode **einmal vor allen Tests** in dieser Klasse aus (muss zur Klasse gehören, also static sein)
- **@BeforeEach:** führt eine Methode **vor jedem einzelnen Test** aus
- **@Order(int num):** legt fest, in welcher Reihenfolge die Tests ausgeführt werden
- **@TestMethodOrder(MethodOrderer.OrderAnnotation.class)** das muss dann über der Klasse stehen

- **assertEquals(expected, Methode)**
- **assertTrue(value)**
- **assertFalse(value)**
- **assertNull(value)**

```
public class Calculator { 4 usages new *  
  
    > public int add(int a, int b) { return a + b; }  
  
    > public int subtract(int a, int b) { return a - b; }  
  
    public int divide(int a, int b) { 1 usage new *  
        if (b == 0) {  
            ⚡ throw new IllegalArgumentException("Division by zero is not allowed!");  
        }  
        return a / b;  
    }  
  
    > public boolean isEven(int n) { return n % 2 == 0; }  
  
}
```

Debugging

Step Over () -> Nächste Zeile (springt NICHT in Methoden)

Step Into () -> Springt IN die Methode hinein

Step Out () -> Springt AUS der Methode heraus

Resume () -> Weiter bis zum nächsten Breakpoint

TODO:

bildet Gruppen und denkt über Aufgabe 1 und 2 nach

Aufgabe 1: *Informationssysteme*

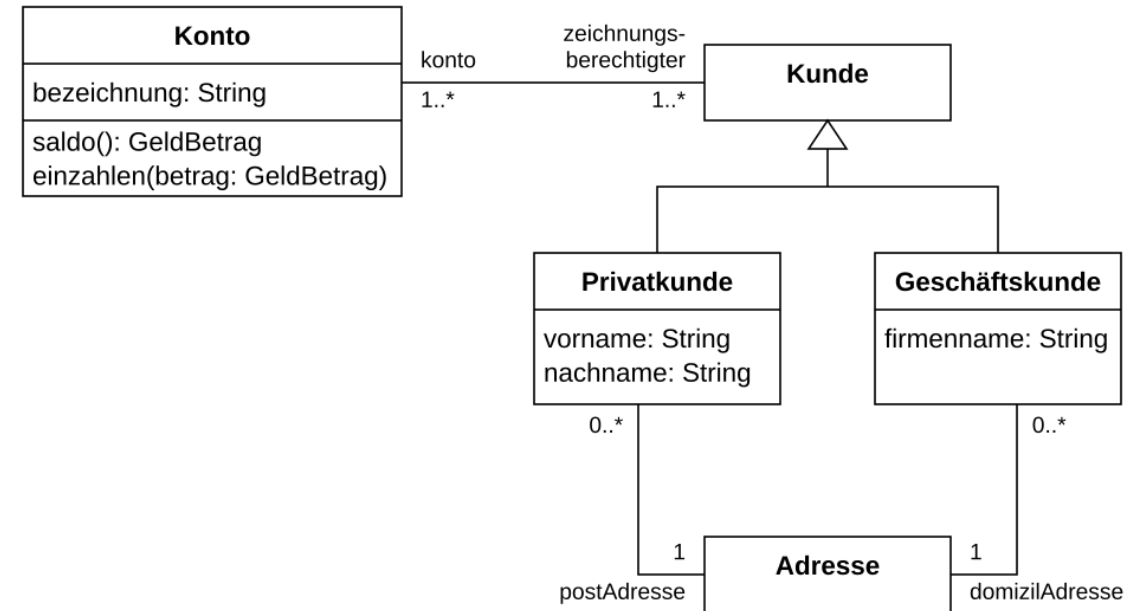
Warum ist die Entwicklung großer Informationssysteme so herausfordernd? Denken Sie dabei an die Beispiele aus der Vorlesung.

- Große Projekte
- Sollte Langlebigkeit sein
- Bootom up design
- Hohe Anforderungen

Aufgabe 2: Softwarearchitektur

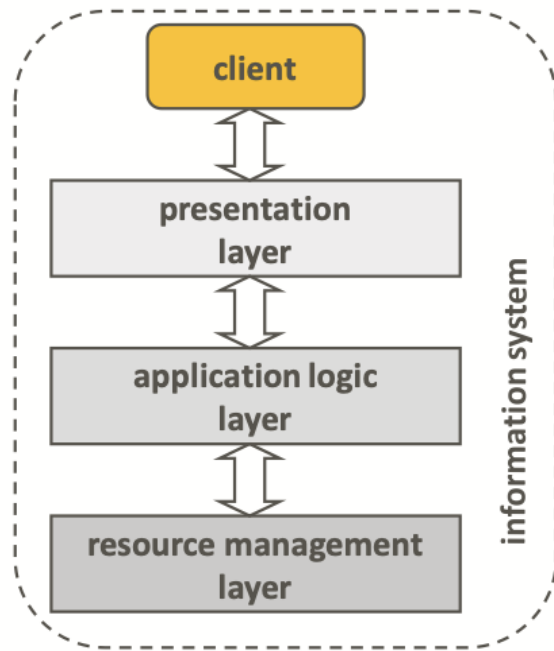
1. Was verstehen Sie unter dem Begriff Softwarearchitektur?
2. Gibt es **die eine** Softwarearchitektur, die besonders gut oder schlecht ist?

- Softwarearchitektur:
- Wie spielen Softwarekomponenten zusammen
- Man schaut auf sie aus mehreren Perspektiven
- lines and boxes (UML Diagramme)
- Zusammenspiel der Komponenten zur Laufzeit

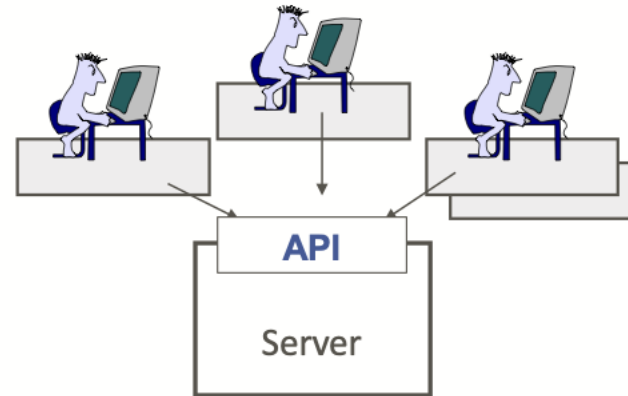


Aufgabe 3 – *tiers* und *layers*

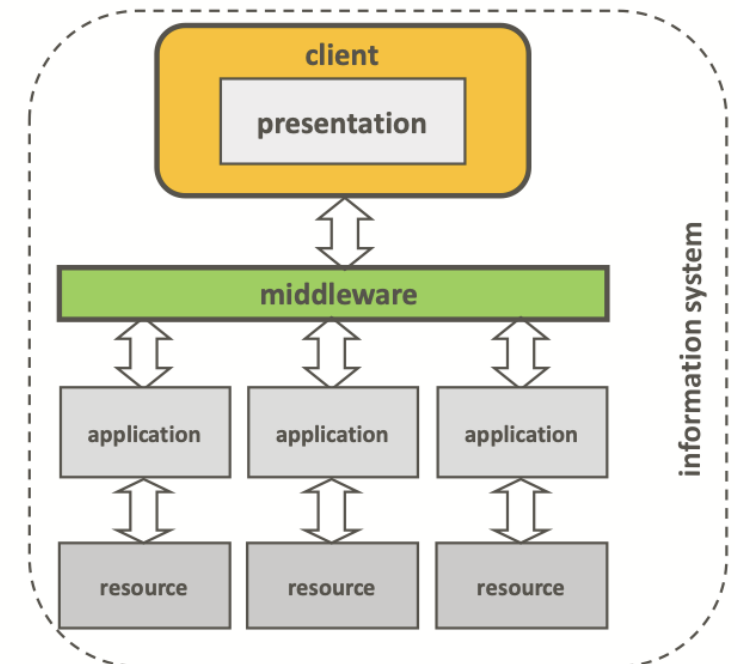
- a) Welche vier Schichten (*layers*) gibt es üblicherweise in einem Informationssystem?
- b) Was ist der Unterschied zwischen *layers* und *tiers*?



2-tier systems / client-server systems



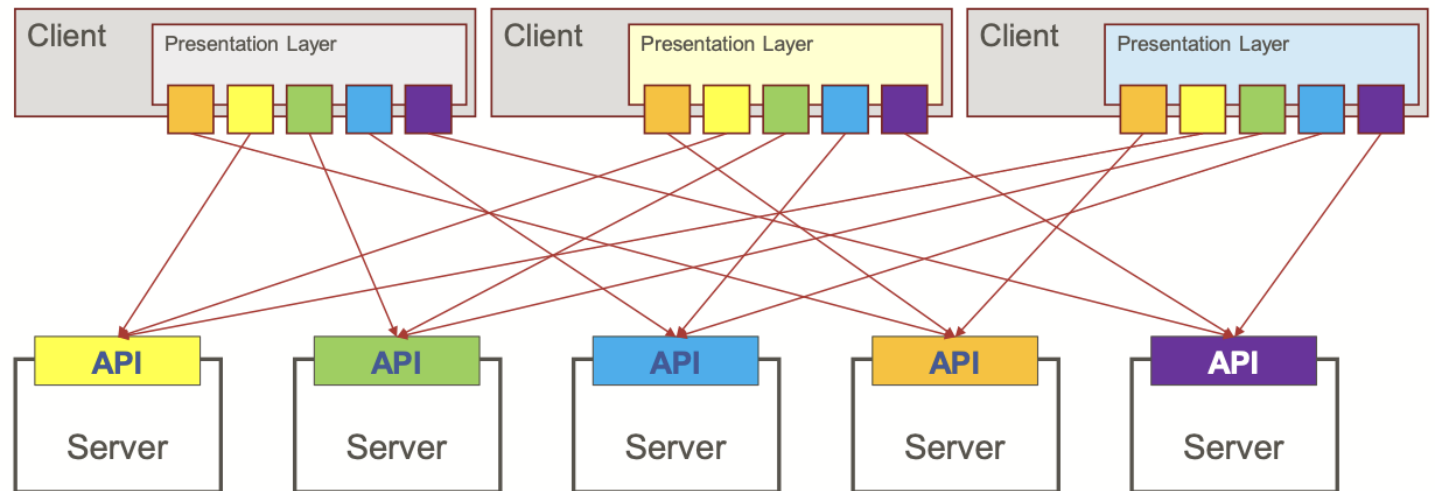
3 tier system:



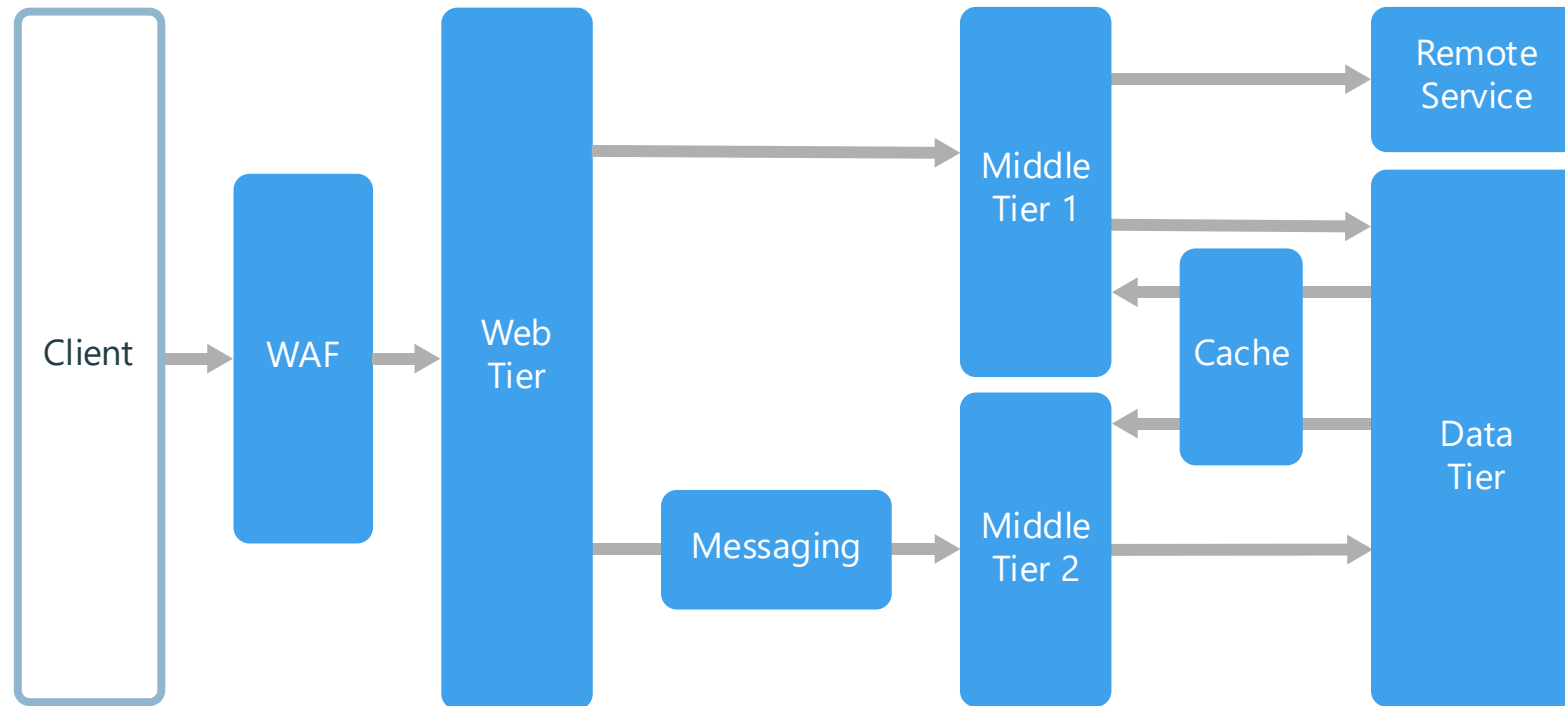
Eine **API** (Application Programming Interface) ist eine digitale Schnittstelle, die es zwei verschiedenen Softwareanwendungen ermöglicht, miteinander zu kommunizieren und Daten auszutauschen.

Ohne Middleware

Limitations of the 2-tier architecture (cont.)



N-tier



Top-down vs. bottom-up design



vs.



Wofür IDLs wenn Bottom up?

- Haben veraltetes System
- alte Datenbanken, Fremdsysteme, APIs, Legacy-Code, Bibliotheken, Betriebssysteme
- Setzen neue, saubere Schnittstelle vor altes System
- -> wollen auch nicht Wissen in Welcher Sprache das System geschrieben wurde
- -> IDLs
- Z.b Protocol buffers

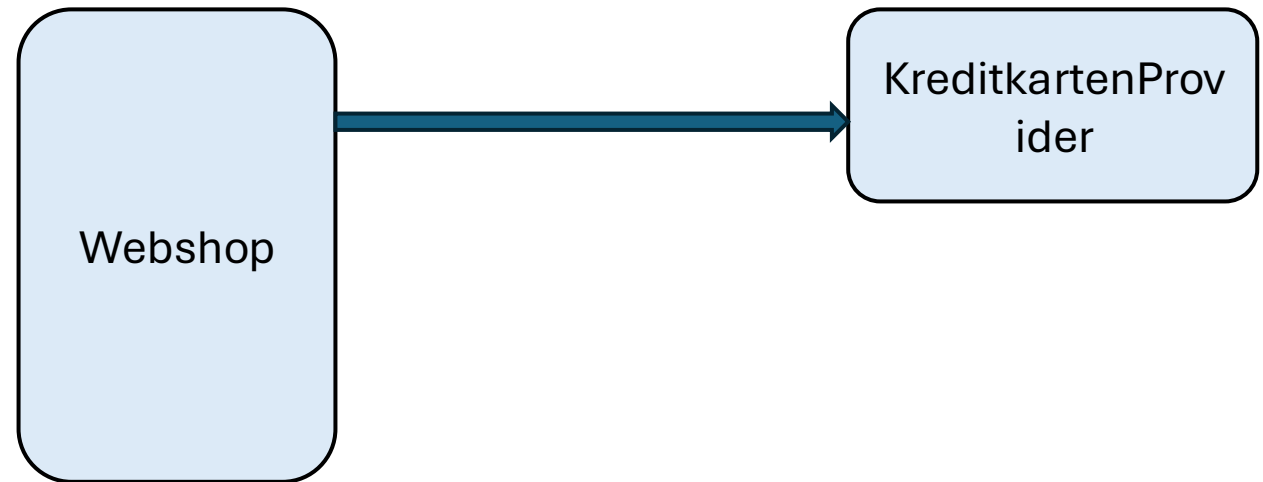
Eine **IDL** (Interface Definition Language) ist eine formale Beschreibungssprache, mit der Softwareschnittstellen sprach- und plattformunabhängig definiert werden

c) Wie beurteilen Sie die folgende Aussage?

Es gibt kein Problem bei der Systemgestaltung, das sich nicht durch Hinzufügen einer Indirektionsebene lösen ließe. Es gibt kein Leistungsproblem, das sich nicht durch Entfernen einer Indirektionsebene lösen ließe.

d) Warum ist das Top-down-Design von Softwarearchitekturen meistens kaum möglich? Welche Probleme ergeben sich dadurch bei der Weiterentwicklung von Systemen bzw. beim Hinzufügen neuer Komponenten?

- **Hinzufügen Abstraktionslevel:** auf jedem level müssen nur noch kleinere Teilprobleme gelöst werden -> parallelität
- -> aber Request muss dann auch alle Abstraktionslevel durchlaufen
- -> hoher Call-Stack
- -> Performance leidet

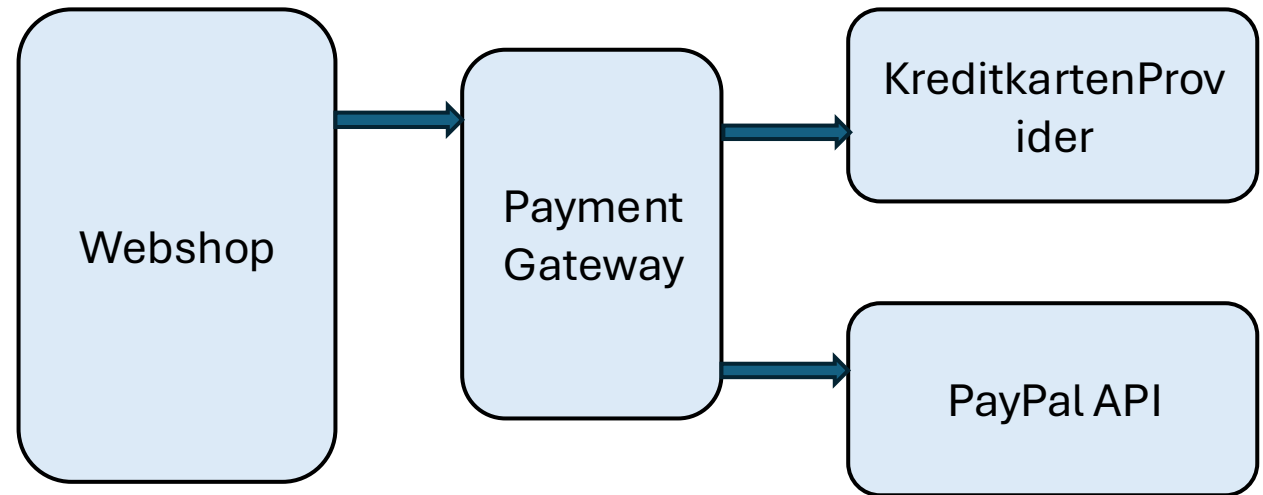


c) Wie beurteilen Sie die folgende Aussage?

Es gibt kein Problem bei der Systemgestaltung, das sich nicht durch Hinzufügen einer Indirektionsebene lösen ließe. Es gibt kein Leistungsproblem, das sich nicht durch Entfernen einer Indirektionsebene lösen ließe.

d) Warum ist das Top-down-Design von Softwarearchitekturen meistens kaum möglich? Welche Probleme ergeben sich dadurch bei der Weiterentwicklung von Systemen bzw. beim Hinzufügen neuer Komponenten?

- **Hinzufügen Abstraktionslevel:** auf jedem level müssen nur noch kleinere Teilprobleme gelöst werden -> parallelität
- -> aber Request muss dann auch alle Abstraktionslevel durchlaufen
- -> hoher Call-Stack
- -> Performance leidet

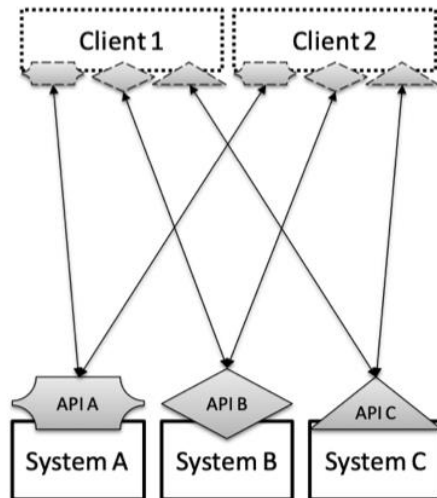


5 – 10 min zeichnen -> Gruppen finden und vergleichen

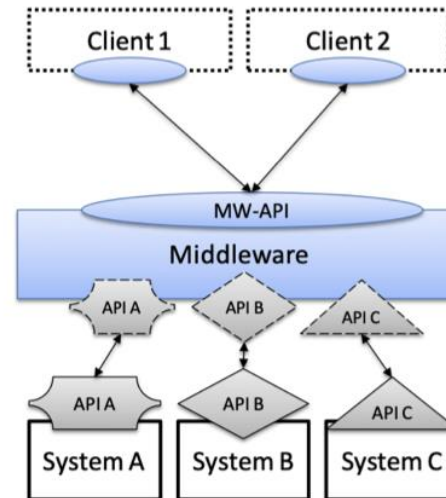
Aufgabe 4 – 2-/3-tier Architekturen

Angenommen, Sie hätten drei vorhandene IT-Systeme (A, B und C), die für diverse Businessaufgaben zuständig sind und die keine standardisierten Schnittstellen besitzen. Ihre Aufgabe ist es, Clients einen einheitlichen Zugriff zu diesen Systemen zu ermöglichen. Geben Sie für diese Problemstellung skizzenhafte Architekturbilder an, aus denen der Unterschied zwischen 2- und 3-tier Architekturen ersichtlich wird. Welche würden Sie in diesem Fall bevorzugen?

2-Tier



3-Tier



Funktional vs Nicht-Funktional

- Funktional:
 - Konkrete features
 - Abhakbar
- Nicht-Funktional:
 - Qualitätsmerkmale, messbar
 - Wie gut? Wo? Unter welchen Bedingungen ?

Aufgabe 6 – Anforderungen

Besprechen Sie zunächst den Unterschied zwischen funktionalen und nichtfunktionalen Eigenschaften. Ordnen Sie die folgenden Anforderungen den Kategorien „funktional“ und „nichtfunktional“ zu. Begründen Sie Ihre Entscheidungen. Ist eine klare und eindeutige Zuordnung immer möglich und sinnvoll?

- | | |
|--|-------|
| 1. Die Anwendung muss auch mit 1000 gleichzeitigen User:innen funktionieren. | 1. Nf |
| 2. Das System muss den Namen, E-Mail und Passwort der User:innen speichern. | 2. F |
| 3. Das System muss mit allen gängigen Browsern funktionieren. | 3. Nf |
| 4. Das System muss alle Anfragen innerhalb von fünf Sekunden beantworten. | 4. Nf |
| 5. Die Software muss ein Lied komponieren können, dass die User:innen beruhigt oder motiviert. | 5. f |
-
- | | |
|--|---------------|
| 6. Die Benutzer:innenoberfläche soll die Marke des Unternehmens widerspiegeln. | 6. Empfehlung |
| 7. Die Datenübertragung zwischen Client und Server muss verschlüsselt und gegen Manipulation abgesichert sein. | 7. F |
| 8. Die Daten der User:innen müssen vor Hackingangriffen geschützt sein. | 8. nf |
| 9. Die User:innen müssen ihre bevorzugte Sprache auswählen können. | 9. f |

5 min

Funktional vs Nicht-Funktional

- **Funktionale Anforderungen (WAS):**

- 2. Daten speichern
- 5. Lied komponieren
- 7. Datenintegrität sichern
- 9. Sprache wählen

- **Nicht-funktionale Anforderungen (WIE GUT):**

- 1. 1000 gleichzeitige User
- 3. Auf allen Browsern
- 4. Antwort in 5 Sekunden
- 8. Vor Hacking schützen

Aufgabe 7 – Leistung

Was ist der Unterschied zwischen Latenz und Durchsatz? Fällt Ihnen ein Beispiel für eine Anwendung ein, bei der Unterschied deutlich wird?

Latenz:

- wie lange ein System braucht um eine request zu verarbeiten
- In ms

Durchsatz:

- Anzahl der requests, die in einer bestimmten Zeit abgearbeitet werden können
- Meistens requests per second

- BSP: Netflix
- **Latenz:** wie lange braucht das Video bis es startet z.b. 50 ms
- **Durchsatz:** Wie viele Daten pro Sekunde?
- 25 Mb/s -> 4k, weil viele Daten

10 min Aufgabe lösen:

Aufgabe 8 – Datenübertragung

Stellen Sie sich das folgende Szenario vor. Auf Ihrem Computer befindet sich ein Verzeichnis, das 10 TB³ Daten enthält. Sie wollen dieses nun aus Berlin an einen Computer senden, der in Hamburg steht. Ihre Internetverbindung ermöglicht das Senden von Daten mit einer Rate von 1000 Mb/s, der Computer in Hamburg kann Daten mit 500 Mb/s empfangen.⁴ Mithilfe des Programms „ping“ haben Sie herausgefunden, dass die Paketumlaufzeit (engl. *round-trip time*) 1000 ms beträgt, wobei Sie davon ausgehen können, dass die Geschwindigkeit in beide Richtungen gleich ist.

- a) Finden Sie heraus, wie groß Latenz und Durchsatz der Datenübertragung sind. Wie lange wird es dauern, bis alle Daten vollständig übertragen wurden?
- b) Eine Kollegin schlägt Ihnen vor, die Daten auf eine Festplatte zu kopieren und direkt nach Hamburg zu fahren. Ihre Festplatte hat Schreib- und Lesegeschwindigkeiten von 1000 Mb/s, die Fahrt nach Hamburg dauert etwa drei Stunden. Was halten Sie von dem Vorschlag?

Info: mb/s -> Megabit

1 Byte = 8 Bit

1 TB = 1 000 000 MegaByte

GitHUb –
Code kommt
später

